

VLSI Verification Lab

Mohammad Attari & William Marnfeldt

1 Introduction

In the grand art of verification, you will commonly face the somewhat difficult task of verifying that things are working as they should. In this lab, you will explore some basic concepts in verification. In this lab you will learn some basics in building a test bench in SystemVerilog. As you may not be particularly familiar to SystemVerilog, don't be afraid look up any unfamiliar concepts. It will all make sense! Anyway, let's take a look at what is of most importance in this lab.

- Constraint Random Testing - Lots and lots of testing!
- Coverage - Primarily Functional Coverage.
- Some useful SystemVerilog features.

2 The DUT & the TB

The two most important acronyms you will see in the realm of verification are DUT and TB. Acronyms are a bit scary, so let's quickly refresh our memories. The Design under Testing (DUT) is the thing we are testing and the Test Bench (TB) is the environment we use to do so. Since this lab is about verification, it will mainly concern the TB. In order to build a good TB we will still need to understand our DUT. To keep it a bit simple, the lab has been limited to 3 files we need to familiarize ourselves with. Let's familiarize ourselves with our available files:

```
lab1
├── simple_alu.sv - Our DUT.
├── tb_pkg.sv - A package for reused assets.
└── test_top.sv - Our primary TB environment.
```

2.1 The DUT

The DUT in question is a simple pipelined ALU. This device can perform 5 functions. These are Add, Subtract, Multiply, Divide, Modulo. It uses an output register and can hold the result for multiple clock cycles. As such we expect the result to be delivered after 1 clock cycle.

2.2 The TB

The Test Bench you will implement in the lab is currently quite empty. Fear not, this is why you are here! Throughout the lab you will be familiarized with some basic concepts in designing useful test benches. Before you implement anything, take a look at `test_top.sv`. It is divided into 11 sections. Let's examine what they do.

Test bench Summary

- **Section 0:** Contains the Randomizer class for stimuli generation.
- **Section 1:** Contains the internal signals and variables used by the test bench.
- **Section 2:** Initializes the internal signals and variables.
- **Section 3:** Instantiates the DUT and the Randomizer.
- **Section 4:** Generates a clock signal with a period of 10ns.
- **Section 5:** Contains a task that is used to generate the reset signal. The reset signal gets activated for a certain length after a certain amount of delay has passed (both in clock cycles)
- **Section 6:** A task similar to the previous section but this one generates the start signal.
- **Section 7:** Contains a task that drives the operations as well as input a and b.
- **Section 8:** Contains the functional coverage definitions. You will be tasked with expanding it's contents.
- **Section 9:** Generates the stimuli by calling the appropriate stimulus generator task. You will modify this task later on in order to produce the necessary stimuli.
- **Section 10:** The main entry point of our TB. From here we will call our tasks.

Make sure you familiarize yourself with your environment. Perhaps you have not seen tasks in SystemVerilog before so here is a quick summary. A task in SystemVerilog is very similar to a function. However, it does not need a return statement and may also consume time. As such, they are very useful, allowing you to set up cool sequences.

3 Tasks

3.1 Task 1 - Warming up

This doesn't look too bad. If you try running it, you should however see an error message like in Figure 1. Why is this popping up?

```
# 315 reset():          Reset released
# ** Error: Reset doesn't clear output!
#      Time: 315 ns  Scope: simula_alu tb test
```

Figure 1: Low Resolution Error Message.

The error message is a small example of an assertion in action. It can be found in section 9. The error message may be annoying, but the fact that we can design a test bench that will help us detect errors is very useful. Your task is now to modify the DUT so that the assertion doesn't fail. Take time to acquaint yourself with SystemVerilog.

Task 1:

Modify the DUT so that the assertion doesn't fail.

As some of you can guess, the DUT is still not fully functional at this point, and is sure to present more buggy behavior later. For now, let's focus on improving our TB, so that we can find more bugs.

3.2 Task 2 - Some Random Stimuli!

Although performing addition with 2 preset numbers is a great beginning, it isn't going to take us very far. Let's take a look at how we can generate some stimuli in a smarter way: If you look at section 1, you will find a class called `RANDOMIZER`. Yes, SystemVerilog is object oriented! To allow a variable in a class to be randomized, it should be declared with the `rand` keyword eg. `rand logic a`. This keyword allows the variables to be randomized by calling `.randomize()` method on an object of the class. The variables can then be accessed as you would a member variable in Python or Java. Try it out for yourself!

Task 2:

Call the `randomize()` method on a `RANDOMIZER` object after allowing all of its member variables to be randomized. Don't forget to modify the test case in section 9 to do math with the random variables!

3.3 Task 3 - Functional Coverage

With our new random stimuli in place, we want to make sure that we are producing reasonable stimuli. This is where functional coverage comes in. Let's use it to cover our stimuli. Currently, our cover group `basic_fcov` is quite empty. It will be up to you to populate the cover group with bins and crossings. We will gradually be improving upon the coverage, so for now let's start with creating cover points for all of the inputs.

Task 3:

Create coverage for signals `tb_opcode`, `tb_a`, `tb_b` and `tb_c`.

"Hey! Wait a minute! What kind of task is that?" you're probably thinking. "How can you possibly cover all of the 2^{32} values?" Well, you don't really want to. Instead, make cover points that cover ranges. Can you think of any values that may be particularly interesting? If so, make specific cover points for them as well! The syntax can be tricky, so if you get stuck, try googling. If that doesn't help, your TA can. While you're at it, let's also make some coverage for the output as well.

3.4 Task 4 - Taking a look at our coverage!

Alrighty, so now that we have something to look at, let's stop and analyze it. Before we do that, however, knowing how to check our coverage may be useful. After running the simulation, open the **View** tab, select **Coverage** and enable the **Covergroups** view, as illustrated in figure 2. You should now be able to see your functional coverage in the new panel, resembling figure 3.

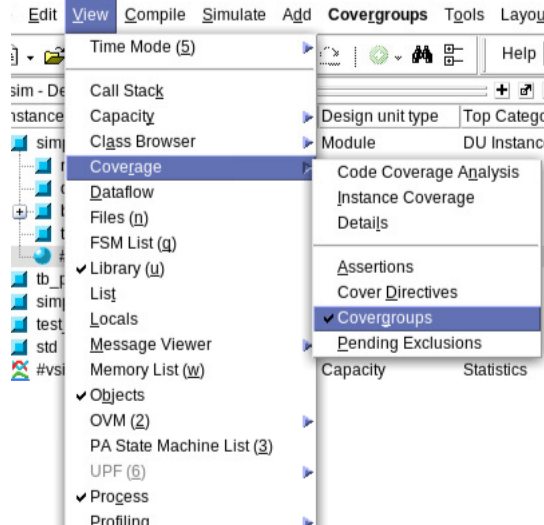


Figure 2: The coverage menu can be found under the View tab.

A screenshot of the 'Covergroups' panel. It displays a table with columns: Name, Class Type, Coverage, Goal, % of Goal, Status, and Included. The table lists various coverage bins for a testbench named 'simple_alu_tb'. The 'TYPE basic_fcov' bin has a coverage of 48.57%. Below it, individual bins for 'CVP basic_fcov:reset', 'CVP basic_fcov:a', 'CVP basic_fcov:b', and 'CVP basic_fcov:op' are listed with their respective coverage percentages. The 'INST Vsimple_alu_tb/coverage_i...' bin also shows a coverage of 48.57%. At the bottom, individual bins for 'CVP reset', 'CVP a', 'CVP b', and 'CVP op' are listed. Each row includes a progress bar and a checkmark indicating the status of the coverage goal.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/simple_alu_tb		48.57%				
TYPE basic_fcov		48.57%	100	48.57%		✓
CVP basic_fcov:reset		100.00%	100	100.00%		✓
CVP basic_fcov:a		40.00%	100	40.00%		✓
CVP basic_fcov:b		40.00%	100	40.00%		✓
CVP basic_fcov:op		14.28%	100	14.28%		✓
INST Vsimple_alu_tb/coverage_i...		48.57%	100	48.57%		✓
CVP reset		100.00%	100	100.00%		✓
CVP a		40.00%	100	40.00%		✓
CVP b		40.00%	100	40.00%		✓
CVP op		14.28%	100	14.28%		✓

Figure 3: Example of coverage bins!

You should be seeing some large holes in the coverage. This is in fact a good thing, as we want to be able to determine the quality of our stimuli. You should be seeing holes in the coverage for inputs a and b, as well as for the Operation type. Let's see if we can't do something about the holes in our coverage. Now that you can see gaps in your coverage, it's time to fix the holes. Start with making sure that each OP is called at least once. Here it may be useful to consider the `randc` keyword. What is the difference between `randc` and `rand`? Note: The `repeat` keyword can also be very useful here to make repeated calls to a task.

Task 4:

Reach full coverage on your input stimuli. If you are using special bins for min/max values, consider using directed calls to cover them.

3.5 Crossings!

Now you can show that you can both produce a large variety of input values as well as use all of the available operators. There is however one common point of failure to be on the lookout for. With your current coverage it is possible to be fooled on what is actually going on. For example, all that is required to get good looking coverage is that every operator is called once, as well as one operator exhausting the numerical range. In other words, it may hide bad coverage for a specific operator. This is where the `cross` keyword comes in. This keyword allows you to cross multiple cover points with regards to each other. In other words, we have a simple way of finding out which values were used for every individual operator! You should have something similar to figure 4.

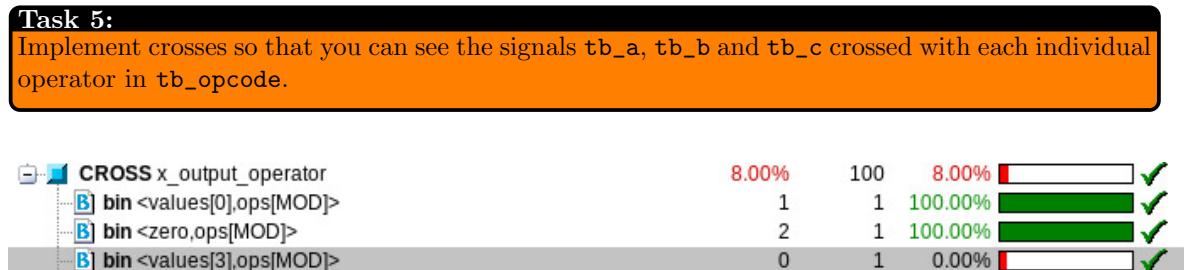


Figure 4: Example of cross coverage in action. Albeit a bit unsorted...

3.6 Random, with a touch of constraint!

With our most excellent stimuli, we are finally ready to look at our DUT. Take a look at the waveforms. Does everything look OK? Can you think of any reasons why we're seeing issues? Although completely random values may seem fun, it is not necessarily a useful practice. In our case we are getting a bunch of overflow scenarios that are ruining our good moods. Let's try to do something about that.

Task 6:
Implement constraints within the Randomizer so that our results never go out of bounds or end up being unexpected. Remember that different operations will need to have different constraints.

Congratulations, everything should be in a lovely state now. You should be able to reach a high degree of coverage. As you can see, getting something basic running wasn't very tricky at all. If you were to continue, you would probably add some sort of automatic checking feature to validate that every input created a valid output. In the upcoming labs you will get to work with more sequential logic, but by now you are probably somewhat comfortable with the basics of random stimuli and coverage. Feel free to experiment further. If you have more time, try designing a class that can be used for checking. There is always more to learn.

Task 7:
When you are finished, call for a TA and present your test bench!